

Test Case — standard format

Copy this file for every new module: `docs/test-cases/TC-<module>.md`.

Two formats live here and they are not interchangeable:

- **Detailed format** — one block per case, with evidence. Use it for a new feature, a complex flow, or any case that will be handed to someone who has not seen the application.
- **Grid format** — one row per case. Use it for a regression sheet where the steps are already understood and the sheet exists to be executed and ticked.

Every case ID follows the suite's own convention — `TC-W<workflow>--<case>`, for example `TC-W10-005` — so a report row, a defect, and a Playwright test title all carry the same ID. Automated cases name their spec file; the test title in that spec **must** begin with the same ID.

Screenshots go in `docs/templates/evidence/` (or `docs/test-cases/evidence/` once copied out) and are embedded inline.

Detailed format

TC-W00-000 — <what the case proves, in one line>

Field	Value
Test case ID	TC-W00-000
Test title	<verb-first — "Search with an exact product name returns that product">
Module / Feature	<Home Products Search Cart Checkout Login API>
Workflow	WF-00 <workflow name from the test case document>
Requirement ID	REQ-000 / user story reference
Test type	Functional UI Negative Boundary Performance Security API E2E
Test level	Smoke Regression Sanity
Priority	High Medium Low
Designed by	<name>
Designed on	YYYY-MM-DD
Automation status	Automated Manual To be automated
Automated in	<code>tests/workflows/wf00-<name>.spec.ts</code>
Tags	<code>@smoke @regression @search</code>

Pre-condition

<State the app must be in before step 1. "None" if the case runs from a cold start as a guest.>

Test data

Field	Value	Source
Search term	<code>top</code>	<code>config/test.config.ts</code> → <code>searchTerms.partial</code>

Field	Value	Source
User	user@example.com	data/users.json

Prefer data read from the catalogue at runtime over a hard-coded value — a content change should never fail a case. Where a literal is unavoidable, name the config key it comes from so it has one home.

Steps

#	Action	Test data	Expected result
1	Open /products	—	The products listing renders; #search_product is visible, enabled and empty
2	Type the search term into #search_product	top	The field holds the typed term
3	Click #submit_search	—	The searched-products view renders
4	Read every product name in the grid	—	At least one result; every name contains top, case-insensitive

Expected result (overall)

<One sentence stating the pass condition for the whole case.>

Actual result

<Filled at execution time.>

Status

Executed by	<name>
Executed on	YYYY-MM-DD
Build	<version / commit sha>
Environment	QA Staging Production
Browser	Chromium / Firefox / WebKit
Result	Pass Fail Blocked Skipped Not Run
Defect ID	BUG-000, or "—"

Evidence

Type	File	Shows
Screenshot	evidence/TC-W00-000-01-step1.png	Starting state
Screenshot	evidence/TC-W00-000-02-result.png	The asserted outcome
Video	evidence/TC-W00-000-run.webm	Full execution

Starting state

PASTE SCREENSHOT HERE

Step 1 — products listing with an empty search field
evidence/TC-W00-000-01-step1.png

Asserted outcome

PASTE SCREENSHOT HERE

Step 4 — every result name contains the search term
evidence/TC-W00-000-02-result.png

Attach evidence for every **Pass** on a new feature and for every **Fail** without exception. A failing case with no screenshot gets re-run instead of fixed.

Post-condition

<State the app is left in. If the case mutates state — adds to cart, creates an account — say how it is cleaned up, and where.>

Grid format — regression sheet

TC ID	Title	Pre-condition	Steps	Test data	Expected result	Priority	Type	Automated	Result	Defect
TC-W10-001	The search control renders on the products page	None	Open <code>/products</code>	—	<code>#search_product</code> visible, enabled, empty; <code>#submit_search</code> visible and enabled	High	UI	Yes		
TC-W10-002	Exact product name returns that product	None	Open <code>/products</code> → search the first catalogue name → read results	First name from the grid	Results contain the term; no irrelevant name	High	Functional	Yes		
TC-W10-005	Surrounding whitespace is trimmed before matching	None	Open <code>/products</code> → search <code>top</code>	<code>top</code>	Same results as <code>top</code>	Medium	Negative	Yes	Fail	BUG-002
TC-W10-00N										

Leave **Result** and **Defect** blank in the committed sheet; they are filled on a run, not at design time.

Worked example — a real case from this suite

TC-W10-005 — Search ignores surrounding whitespace in the term

Field	Value
Test case ID	TC-W10-005
Test title	Searching a keyword padded with spaces returns the same results as the trimmed keyword
Module / Feature	Products — Search
Workflow	WF-10 Product Search
Test type	Negative
Test level	Regression
Priority	Medium
Automation status	Automated
Automated in	<code>tests/workflows/wf10-product-search.spec.ts</code>
Tags	<code>@regression @search</code>

Pre-condition

None — runs as a guest, `storageState: GUEST_SESSION`.

Test data

Field	Value	Source
Trimmed term	<code>top</code>	<code>config/test.config.ts</code> → <code>searchTerms.partial</code>
Padded term	<code>top</code>	The same term, three spaces each side

Steps

#	Action	Test data	Expected result
1	Open <code>/products</code>	—	Listing renders; the search control is empty
2	Search the trimmed term	<code>top</code>	Searched-products view renders with one or more results; record the count
3	Clear the field and search the padded term	<code>top</code>	Searched-products view renders
4	Compare the two result sets	—	Identical count and identical names

The term is passed through unchanged — `ProductsPage.search` must not trim it, or the case stops testing the server and starts testing itself.

Expected result (overall)

A padded term and its trimmed equivalent return the same products.

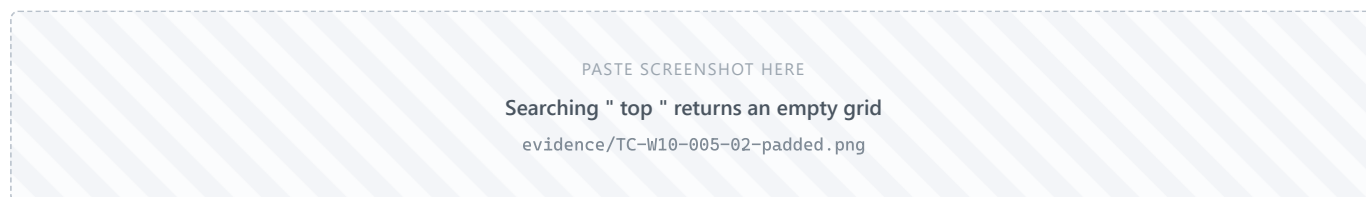
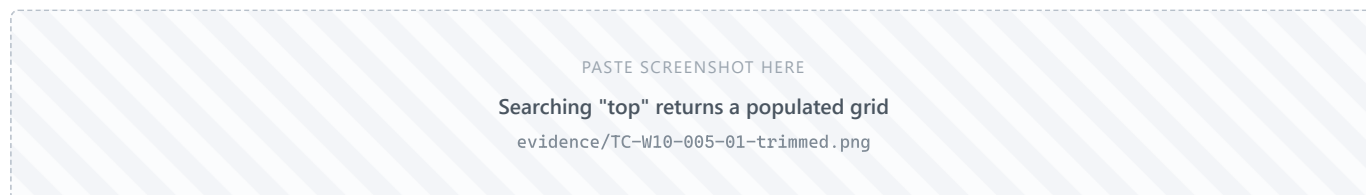
Actual result

The padded term returns zero results, with no message explaining why.

Status

Executed on	2026-10-04
Environment	Production demo
Browser	Chromium
Result	Fail
Defect ID	BUG-002 (D-02 in <code>docs/known-defects.md</code>)

Evidence



Post-condition

No state change — the search is a read. Nothing to clean up.

Field rules, so two people fill these the same way

Field	Rule
Test case ID	<code>TC-W<workflow, 2 digits>-<case, 3 digits></code> . Never reused, never renumbered. A deleted case leaves a gap.
Test title	Verb-first, states the behaviour, not the step. "Returns only relevant results", not "Click search".
Steps	One action per row. Name the element by its locator key, not by how it looks.
Expected result	Observable and checkable. "The grid renders four products", not "It works correctly".
Test data	Prefer runtime data; otherwise name the config key. Never inline a credential — <code>.env</code> only.
Result	Blocked = could not run. Fail = ran, and the expected result did not hold. They are not the same thing.
Defect ID	Required on every Fail. A Fail with no defect ID is an untracked defect.

Traceability

Each case maps one-to-one onto a Playwright test title, so a report row traces straight back to the document. Keep `docs/workflow-traceability.md` in step whenever a case is added, re-numbered, or retired.

```
# Run one case by its ID
npm run test:workflows -- -g TC-W10-005

# Run an entire workflow
npm run test:workflows -- -g WF-10
```